

Rapport technique — SDB (Standalone Docker Backup)

Projet : Outil autonome de sauvegarde et de restauration des volumes Docker

Version : 0.2 — **Date :** juillet 2026

Équipe : Groupe 3 — Kamel ACHOUITAR, Antoine BATTAIRE, Célian DUHEM, Paul LAZARO (ESGI 5 SI)

Stack technique : Go 1.25 · Vue 3 / TypeScript · restic · SQLite · Docker

Sommaire

1. Introduction
 2. Présentation de la solution
 3. Architecture technique
 4. Choix technologiques
 5. Gestion et stockage des données
 6. Sécurité
 7. API, temps réel et supervision
 8. Qualité et industrialisation
 9. Bilan et perspectives
-

1. Introduction

1.1 Contexte

Dans les infrastructures conteneurisées modernes, l'ensemble des données persistantes — bases de données, fichiers utilisateurs, configurations — est stocké dans des **volumes Docker**. Pourtant, Docker ne propose aucun mécanisme natif de sauvegarde : en cas de mauvaise manipulation, d'erreur humaine ou d'attaque, un volume peut être perdu de manière irréversible.

Face à ce risque, de nombreuses équipes recourent encore à des scripts artisanaux (tar + cron) ou à des outils en ligne de commande. Ces solutions manquent d'historique centralisé, de chiffrement systématique et d'une procédure de restauration fiable en situation de crise — c'est-à-dire précisément au moment où l'on en a le plus besoin.

1.2 Objectifs du projet

SDB (*Standalone Docker Backup*) a été conçu pour répondre à ce besoin à travers quatre objectifs majeurs :

Objectif	Traduction concrète
Autonomie	un seul binaire exécutable avec interface web embarquée, sans dépendance externe à installer
Sécurité	chiffrement de bout en bout, conteneur durci, secrets jamais stockés en clair
Simplicité	sauvegarde et restauration en un clic, planification intégrée
Observabilité	progression en temps réel, historique complet, métriques Prometheus

1.3 Périmètre

Le projet permet la découverte automatique des conteneurs et de leurs volumes, la sauvegarde manuelle ou planifiée vers sept types de destinations, la restauration ciblée d'un volume, ainsi que la gestion des utilisateurs et des politiques de rétention. Il est volontairement limité à **un seul hôte Docker** par instance de SDB.

2. Présentation de la solution

2.1 Fonctionnalités principales

- **Sauvegarde à chaud ou à froid** : possibilité d'arrêter temporairement le conteneur pour garantir une cohérence parfaite, avec redémarrage automatique assuré ;
- **Hooks pré/post-sauvegarde** : exécution de commandes personnalisées dans le conteneur cible (ex. `pg_dumpall` avant le snapshot d'une base PostgreSQL) ;
- **Snapshots incrémentaux, dédoublés et chiffrés** grâce au moteur **restic** ;
- **Restauration en un clic** vers le volume d'origine depuis n'importe quel snapshot, avec gestion automatique de l'arrêt et du redémarrage du conteneur ;
- **Planification avancée** : expressions cron 5 champs, activation et désactivation à chaud, déclenchement manuel, suivi de la dernière exécution ;
- **Rétention automatique** : politiques `keep-last` / `daily` / `weekly` / `monthly` appliquées après chaque sauvegarde réussie (`restic forget --prune`) ;
- **Vérification d'intégrité** des dépôts, planifiée (hebdomadaire par défaut) ou à la demande ;
- **Multi-utilisateurs** avec deux rôles : administrateur et utilisateur standard.

2.2 Interface utilisateur

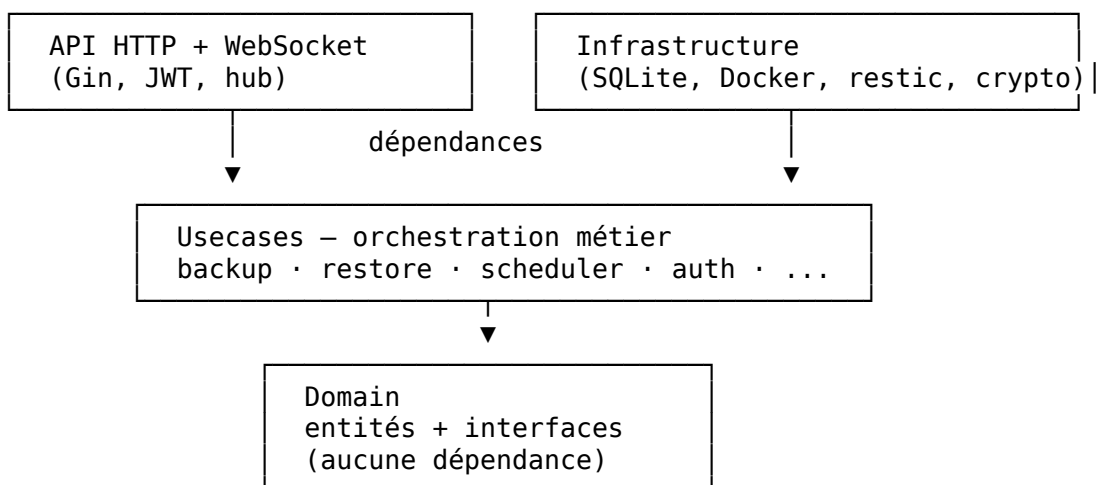
Le tableau de bord web, conçu dans un thème sombre moderne, offre une vision claire de l'état du système via un indicateur global unique (« North Star » : vert / ambre / rouge). Il présente la liste des conteneurs avec leurs actions rapides, le suivi en temps réel des opérations en cours (barres de progression alimentées en WebSocket), l'historique filtrable des sauvegardes et restaurations, ainsi que la gestion des stockages, des planifications et des utilisateurs.

Les formulaires suivent le principe de **divulgaration progressive** : seuls les champs essentiels sont visibles par défaut, les options avancées (hooks, rétention, sélection de volumes) étant repliées.

3. Architecture technique

3.1 Clean Architecture

Le backend repose sur les principes de la **Clean Architecture** : toutes les dépendances pointent vers le domaine métier, jamais l'inverse.

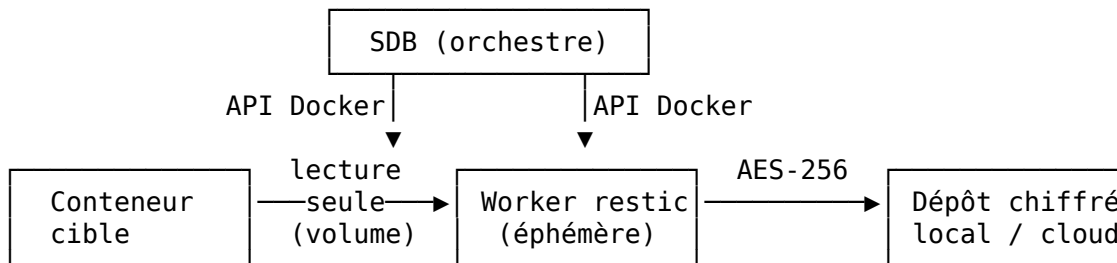


Couche	Répertoire	Rôle
Domain	internal/domain/	entités métier et <i>ports</i> (interfaces)
Usecases	internal/usecase/	règles métier, orchestration des ports
Infrastructure	internal/infra/	adaptateurs SQLite, Docker, restic, crypto
Livraison	internal/api/http/	API REST, WebSocket, authentification

Le point d'entrée `cmd/sdb/main.go` assemble les couches au démarrage (injection de dépendances). Ce découpage permet de tester le cœur métier de manière isolée, **sans Docker ni base de données** : les 63 tests automatisés du projet s'appuient sur des implémentations factices des ports.

3.2 Le worker éphémère

Une décision architecturale centrale : **SDB ne lit jamais directement les données des volumes**. Pour chaque opération, il crée via l'API Docker un conteneur temporaire basé sur l'image officielle `restic/restic`, qui exécute le travail avant d'être systématiquement détruit.



Cette approche garantit :

- le montage des volumes **en lecture seule** dans le worker (sauvegarde) ;
- l'injection des secrets (mot de passe restic, clés cloud) **en mémoire uniquement** — ils disparaissent avec le conteneur ;
- une isolation maximale : pour un dépôt local, le worker tourne **sans aucun accès réseau** ;
- l'absence totale d'écriture sur le disque de l'hôte : les fichiers sensibles (clé SSH, compte de service Google) sont copiés dans le worker par l'API Docker en permissions `0600`.

3.3 Pipeline d'une sauvegarde

L'API répond immédiatement par un `202 Accepted` ; le traitement s'effectue dans une goroutine dédiée et annulable, dont la progression est diffusée en temps réel via WebSocket :

1. Exécution du **pre-hook** dans le conteneur cible — par défaut, un échec annule la sauvegarde : mieux vaut pas de snapshot qu'un snapshot incohérent ;
2. **Arrêt optionnel** du conteneur (sauvegarde à froid) ;
3. Création du **snapshot** via le worker éphémère ;
4. **Redémarrage garanti** du conteneur : cette étape s'exécute sur un contexte insensible à l'annulation — quoi qu'il arrive (erreur, crash, annulation), le conteneur de production repart ;
5. Exécution du **post-hook** (nettoyage) — un échec produit un avertissement, pas un échec de la sauvegarde ;
6. Application de la **politique de rétention**, uniquement en cas de succès.

Des verrous garantissent qu'une seule sauvegarde s'exécute à la fois par conteneur, et une seule restauration à la fois par volume. Enfin, au redémarrage de SDB, les opérations interrompues par un arrêt brutal sont automatiquement marquées en échec.

3.4 Frontend

L'interface Vue 3 (Composition API, TypeScript strict) est compilée puis **embarquée directement dans le binaire Go** via `go:embed` : en production, un seul exécutable sert l'API et l'interface. L'état global est géré par Pinia (session, santé du système, flux d'événements), et les types TypeScript reflètent fidèlement les DTO du backend, ce qui fige le contrat d'API côté client.

4. Choix technologiques

Technologie	Alternatives écartées	Justification
Go	Node.js, Python	binaire statique unique, concurrence native (goroutines), typage fort
restic	moteur maison, tar	solution mature et auditée offrant chiffrement, déduplication et incrémental ; réinventer un moteur de sauvegarde aurait été le principal risque du projet
SQLite (pur Go)	PostgreSQL, driver CGO	zéro dépendance externe, compilation statique, largement suffisant pour des métadonnées
Vue 3 + TypeScript	React	Composition API concise, typage strict vérifié en CI
Gin	net/http seul	routage, binding JSON et middlewares matures
Image distroless	Alpine, Debian	pas de shell ni de gestionnaire de paquets dans l'image finale : surface d'attaque minimale

Toutes les dépendances sont verrouillées (`go.sum`, `package-lock.json`) et les builds sont reproductibles.

5. Gestion et stockage des données

5.1 Trois niveaux de persistance

Type de données	Emplacement	Protection
Métadonnées (comptes, historiques, planifications, configurations)	SQLite, volume Docker /data	secrets chiffrés AES-256-GCM sous la clé maître
Sauvegardes (contenu des volumes)	dépôts restic : disque local, S3, Backblaze B2, Azure Blob, Google Cloud Storage, SFTP, serveur REST	chiffrement de bout en bout par restic (AES-256), déduplication
Secrets d'exploitation (clé maître, secret JWT)	fichiers montés (Docker secrets)	jamais en clair dans le code, la base ou le dépôt Git

5.2 Modèle de données

Cinq tables principales, avec migrations SQL versionnées et embarquées dans le binaire (appliquées automatiquement au démarrage, transactionnelles, avec contrôle d'intégrité référentielle) :

- `users` — comptes (hash Argon2id, rôle) ;
- `storage_configs` — destinations (identifiants et mot de passe restic stockés en blobs chiffrés) ;
- `backups_history` / `restores_history` — journal de chaque opération (statut, octets traités, snapshot, horodatages, journal d'erreur) ;
- `backup_schedules` — planifications (cron, options, dernière exécution).

5.3 Point critique : la clé maître

La clé maître (SDB_MASTER_KEY) protège les configurations de stockage au repos. Sa perte rend ces dernières indéchiffrables — elle doit donc être sauvegardée séparément des données. Les sauvegardes restic restent en revanche restaurables directement avec le mot de passe du dépôt.

6. Sécurité

La sécurité a constitué la priorité absolue du projet, avec une logique de défense en profondeur à chaque couche.

Authentification et comptes

- hachage des mots de passe avec **Argon2id** (recommandations OWASP), comparaison en temps constant, protection contre l'énumération de comptes ;
- jetons **JWT HS256 avec algorithme épinglé** (bloque l'attaque par confusion d'algorithme), *rate-limiting* du login (10 tentatives/min/IP) ;
- invariant métier : impossible de supprimer ou de rétrograder le dernier administrateur.

Gestion des secrets

- chiffrement **AES-256-GCM** (authentifié) de tous les secrets au repos, nonce aléatoire par opération ;
- mot de passe de dépôt restic **généré automatiquement** (43 caractères) et **immuable** — le modifier verrouillerait le dépôt.

Durcissement du conteneur et du réseau

- rootfs en **lecture seule, cap_drop ALL, no-new-privileges**, /tmp en tmpfs, image distroless ;
- port publié **uniquement sur 127.0.0.1** — la publication de port Docker contourne UFW/iptables, ce piège est neutralisé par construction ;
- connexion tcp:// au démon Docker **refusée au démarrage** sans mTLS complet ;
- WebSocket protégé par une vérification d'origine *same-origin* stricte.

Le risque assumé : SDB monte le socket Docker, ce qui équivaut à un accès root sur l'hôte. C'est inhérent à sa fonction d'orchestrateur ; l'ensemble du durcissement ci-dessus vise à ce que SDB lui-même ne devienne pas un point d'entrée.

7. API, temps réel et supervision

7.1 API REST

31 endpoints sous /api/v1, protégés par JWT (à l'exception du login), avec autorisation par rôle sur les mutations sensibles. Les opérations longues retournent un 202 Accepted et se suivent en temps réel. Les erreurs métier se projettent systématiquement sur les codes HTTP appropriés (400, 401, 403, 404, 409, 503) ; les erreurs inattendues renvoient un 500 opaque, le détail restant dans les journaux.

Famille	Exemples
Authentification	POST /auth/login
Conteneurs	GET /containers, GET /health
Stockages	CRUD + GET /storage/:id/snapshots, POST /storage/:id/check
Sauvegardes	POST /backups → 202, annulation, historique
Restaurations	POST /restores → 202, annulation, historique
Planifications	CRUD + POST /schedules/:id/run
Utilisateurs	CRUD (admin), changement de mot de passe

7.2 Temps réel : le hub WebSocket

Un *hub* central diffuse les événements de progression à tous les navigateurs connectés, avec une propriété essentielle : **il ne bloque jamais les opérations métier**. Une seule goroutine possède la liste des clients (aucun verrou); un client trop lent est déconnecté plutôt qu'attendu; un hub saturé abandonne l'événement, l'état faisant foi étant en base. Côté client, la reconnexion est automatique avec *backoff* exponentiel (1 s → 30 s).

7.3 Supervision : Prometheus

L'endpoint `/metrics` (désactivé par défaut, protégé par un jeton statique) expose notamment `sdb_backups_total{status}`, `sdb_running_jobs` et `sdb_last_backup_success_timestamp_seconds` — cette dernière métrique permet l'alerte la plus utile en production : « aucune sauvegarde réussie depuis 24 h pour ce conteneur ». Le collecteur consomme le même flux d'événements que le hub WebSocket : son ajout n'a modifié aucune ligne de logique métier.

8. Qualité et industrialisation

8.1 Tests

63 tests automatisés (environ 2 000 lignes), exécutés avec le détecteur de concurrence de Go. Grâce à l'architecture en ports et adaptateurs, le pipeline complet de sauvegarde est testé **sans démon Docker** : rollback après échec, politiques des hooks, conflits d'exécution simultanée, annulation, non-fuite des secrets (vérification qu'aucun secret en clair n'apparaît dans le fichier SQLite), sécurité JWT (rejet d'un jeton `alg=none`) et éviction des clients WebSocket lents.

8.2 Validation en conditions réelles

Le cycle complet a été validé de bout en bout sur un démon Docker réel : planification cron déclenchée à la seconde près, politique de rétention vérifiée (15 sauvegardes ramenées à 3 snapshots conservés), et scénario complet *sauvegarde* → *suppression des données* → *restauration à l'identique*, avec vérification des contenus et des horodatages.

8.3 CI/CD et déploiement

- **GitHub Actions** en trois jobs : backend (go vet, tests avec race detector, build), frontend (vue-tsc strict, build Vite) et construction de l'image Docker complète;
- image **multi-stage** légère (~35 Mo) : build du frontend, compilation Go statique avec le frontend embarqué, image finale distroless;
- déploiement en une commande : `docker compose up -d --build` — migrations et compte administrateur initial créés automatiquement au premier démarrage.

8.4 Chiffres clés

Indicateur	Valeur
Lignes de code (Go + TypeScript/Vue, hors tests)	≈ 8 800
Lignes de tests	≈ 2 000
Tests automatisés	63
Endpoints API	31
Destinations de stockage supportées	7
Taille de l'image finale	≈ 35 Mo

9. Bilan et perspectives

9.1 Bilan

SDB est un outil **complet, fonctionnel et sécurisé**, validé en conditions réelles, qui remplit pleinement ses objectifs initiaux : autonome (un binaire, une commande de déploiement), sécurisé (chiffrement systématique et défense en profondeur), simple (sauvegarde et restauration en un clic) et observable (temps réel, historique, métriques).

Au-delà des fonctionnalités, le principal acquis est architectural : l'association de la Clean Architecture et du worker éphémère produit un système dont le cœur métier est testé isolément, dont chaque brique technique est remplaçable, et dont l'outil de sauvegarde n'a jamais accès en écriture aux données qu'il protège.

9.2 Limites actuelles

- limitation à un seul hôte Docker par instance ;
- absence de notifications proactives en cas d'échec (l'information reste disponible dans l'historique et les métriques) ;
- TLS à assurer via un reverse proxy pour un accès distant.

9.3 Perspectives

1. Notifications d'échec par e-mail et webhook ;
2. Support multi-hôtes (le mTLS vers des démons distants est déjà supporté) ;
3. TLS natif sur l'interface ;
4. Publication open source du projet.

Document rédigé par le Groupe 3 (ESGI 5 SI) dans le cadre de la soutenance du projet SDB — juillet 2026.